Central Board of Secondary Education

New Delhi

&

Delhi Public School, Fulbari

Affiliated with Central Board of Secondary Education, New Delhi

Affiliation No.: **2430269** – School Code: **15908**

Address: CHHOBAVITA, CANAL ROAD, P.O. FULBARI, BHAKTI NAGAR DIST: JALPAIGURI

PIN-734015, (WEST BENGAL)

PROJECT FILE

Project Title: **Smart School Canteen: Food Waste Analysis and Demand Planning**

Subject: **Informatics Practices (065)**

Submitted to: **Department of Computer Science, DPS Fulbari**

For the partial fulfilment of AISSCE Examination for the Academic Session: 2026-27

SUBMITTED BY

Name: _____

Class: **XII**

Stream: _____

Board Roll No.: _____

Sample Copy Prepared by: Bikramaditya Bagchi PGT (CS&IP) © 2026

ID SHEET

Board Roll No.: _____

Name of the Student: _____

Supervisor: **Bikramadittya Bagchi**

Project Title: **Smart School Canteen: Food Waste Analysis and Demand Planning**

Front End Tool: Python 3.13

Backend Tool:

Sample Copy Prepared by: Bikramadittya Bagchi PGT (CS&IP) © 2026

CERTIFICATE

This is to certify that _____ of Class XII, _____ of Delhi Public School, Fulbari, Siliguri has done the project titled on **Smart School Canteen: Food Waste Analysis and Demand Planning** under my supervision.

The student has taken a keen interest and has shown utmost sincerity in completing the project.

I certify that this project is up to my expectation and as per the guidelines issued by **Central Board of Secondary Education, New Delhi.**

_____ _____

Internal Examiner **External Examiner**

_____

Principal

Sample Copy Prepared by: Bikramaditya Bagchi PGT (CS&IP) © 2026

ACKNOWLEDGEMENT

It is with pleasure that I acknowledge my sincere gratitude to our teacher, Bikramadittya Bagchi, PGT (CS & IP) who taught and undertook the responsibility for teaching the subject Informatics Practices. I have greatly benefited from his classes.

I am especially indebted to our Principal, Mrs. Monowara B. Ahmed, who has always been a source of encouragement, support and without whose inspiration, this project would not have been successful.

Finally, I would like to express my sincere appreciation for all other students of my batch for their support, friendship & the fine time we all shared together.

_____

Signature

Dated:

Sample Copy Prepared by: Bikramadittya Bagchi PGT (CS&IP) © 2026

CONTENTS

Sl.	Name	Page
1.	ID Sheet	i
2.	Certificate	ii
3.	Acknowledgement	iii
4.	Introduction	2
5.	…….. Continue…….	

Sample Copy Prepared by: Bikramaditya Bagchi PGT (CS&IP) © 2026

INTRODUCTION

Food waste is a common problem in school canteens. Every day, canteens prepare different food items according to the expected demand of students and staff. However, the actual quantity sold may be different from the quantity prepared. When more food is prepared than required, the remaining food may go to waste. On the other hand, preparing less food than required can result in popular food items becoming unavailable.

The Smart School Canteen project is designed to analyse food preparation, sales, leftovers, and associated costs with the help of computer-based data analysis. The project uses Python as the programming language, Pandas for data manipulation, Matplotlib for graphical representation, and CSV files for storing records.

The program maintains information about the date, food item, quantity prepared, quantity sold, cost per portion, and selling price per portion. Using this information, it calculates the quantity of food left over, waste percentage, estimated waste cost, sales revenue, preparation cost, and estimated net result.

The project also generates daily summaries and food-item-wise reports to identify items that produce the most waste. Based on previously recorded sales, it recommends an approximate preparation quantity for each food item. This recommendation uses average recorded daily sales with an additional 10 percent buffer.

The program provides a menu-driven interface through which users can add records, view existing data, search for specific records, analyse food waste, generate demand plans, export reports, and view graphical representations of the results.

The main aim of this project is to demonstrate how the concepts of Informatics Practices can be applied to a real-world problem. It encourages the use of data for better decision-making, cost control, and responsible resource management in school canteens.

Sample Copy Prepared by: Bikramaditya Bagchi PGT (CS&IP) © 2026

EVOLUTION AND NEED FOR THE PROJECT

Traditionally, school canteens may maintain daily records of food preparation and sales in notebooks or registers. Although this method is simple, analysing a large number of records manually can be time-consuming and may lead to calculation errors. It can also be difficult to identify long-term patterns in food demand and wastage.

With the increasing availability of computers and data-processing tools, manual record keeping can be improved through digital systems. Digital records allow information to be stored systematically, retrieved easily, and analysed whenever required.

The development of the Smart School Canteen project follows this approach. It converts routine canteen information into structured data and uses Python libraries to generate meaningful summaries and visual reports.

The project is needed for the following reasons:

1. **Reduction of food waste:** Recording the quantity prepared, sold, and left over helps identify food items that are repeatedly produced in excess.
2. **Better demand estimation:** Analysing past sales helps canteen staff estimate how much of each food item may be required in the future.
3. **Cost monitoring:** Calculating the estimated cost of leftover food helps identify the financial value of resources that are not being used effectively.
4. **Improved record management:** Storing records in a CSV file makes them easier to maintain, update, transfer, and analyse.
5. **Faster analysis:** Pandas performs calculations and generates summaries more efficiently than repeated manual calculations.
6. **Visual understanding of data:** Matplotlib charts help users compare food items, understand sales trends, and recognise patterns in food wastage.
7. **Evidence-based decisions:** The project provides information that can support decisions about food preparation quantities, menu planning, and resource utilisation.

Thus, the project represents a shift from basic record keeping towards data-driven planning. It is intended to support better decisions rather than replace the judgement and experience of canteen staff.

OBJECTIVES OF THE PROJECT

The primary objective of the Smart School Canteen project is to use Python-based data analysis to understand food wastage and improve demand planning in a school canteen.

The specific objectives are:

1. To create a simple, menu-driven application for managing school canteen records.
2. To store daily food preparation and sales information in a CSV file.
3. To use Pandas DataFrames to read, organise, filter, process, and analyse canteen data.
4. To calculate the quantity of leftover food by subtracting the quantity sold from the quantity prepared.
5. To calculate the percentage of food left over in relation to the total quantity prepared.
6. To estimate the cost of leftover food using the cost of preparing one portion.
7. To calculate sales revenue, total preparation cost, and an estimated net result for the recorded food items.
8. To generate daily summaries and food-item-wise summaries for easier comparison.
9. To identify food items with high leftover quantities or high estimated waste costs.
10. To estimate future preparation quantities using average recorded daily sales and an additional 10 percent buffer.
11. To represent the analysed information graphically using Matplotlib bar charts and line graphs.
12. To export daily summaries, food-item summaries, and demand-planning reports into separate CSV files.
13. To apply input validation and data checks to reduce errors during record entry and processing.
14. To demonstrate the practical use of Python, Pandas, Matplotlib, and CSV files in solving a real-world problem.

Sample Copy Prepared by: Bikramaditya Bagchi PGT (CS&IP) © 2026

KEY CODE FUNCTIONALITIES INTEGRATED

The project combines several Python programming concepts and library functions to provide a complete data analysis solution.

- **CSV File Creation and Data Loading**

The program uses a CSV file named canteen_data.csv to store the canteen records. If the file does not exist, the program creates an empty CSV file with the required column headings.

The pandas.read_csv() function reads the records into a DataFrame for processing.

This approach allows the records to remain available after the program is closed.

- **Input and Data Validation**

The program validates user input before storing new records. It checks that dates follow the required format, food item names are not blank, quantities are valid whole numbers, and monetary values are non-negative.

It also checks that the quantity sold does not exceed the quantity prepared and that the quantity prepared is greater than zero.

These checks help prevent incorrect records from affecting the analysis.

- **Adding, Viewing, and Searching Records**

The program allows users to add new canteen records, display existing records, and search for records by date or food item name.

The search functionality uses Pandas filtering to display only the records that match the entered conditions. This makes it easier to retrieve information without examining the entire dataset manually.

- **Calculation of Leftover Quantity**

The leftover quantity is calculated by subtracting the quantity sold from the quantity prepared.

Formula:

Leftover Quantity = Quantity Prepared - Quantity Sold

For example, if 100 portions are prepared and 85 portions are sold, the leftover quantity is 15 portions.

- **Calculation of Waste Percentage**

The waste percentage indicates the proportion of prepared food that remains unsold.

Formula:

$$\text{Waste Percentage} = \left(\frac{\text{Leftover Quantity}}{\text{Quantity Prepared}}\right) \times 100$$

For example, if 15 portions remain out of 100 portions prepared, the waste percentage is 15 percent.

This measure helps compare wastage between food items with different preparation quantities.

- **Financial Analysis**

The project calculates several financial measures from the recorded quantities and prices.

Estimated Waste Cost = Leftover Quantity x Cost Per Portion

Sales Revenue = Quantity Sold x Selling Price Per Portion

Preparation Cost = Quantity Prepared x Cost Per Portion

Estimated Net Result = Sales Revenue - Preparation Cost

These calculations help the user understand the approximate financial effect of food preparation and sales.

The estimated net result is a simplified measure. It does not include other expenses such as electricity, labour, rent, or transportation.

- **Daily Summary and Food-Item Summary**

The program uses the Pandas groupby() and agg() functions to group data and calculate totals.

The daily summary displays quantities prepared, sold, and left over on each recorded date, along with relevant financial calculations.

The food-item summary compares different food items using total preparation, total sales, leftover quantities, waste percentages, and estimated waste costs.

These reports help identify patterns that may not be obvious from individual records.

♦ **Food Waste Analysis**

The food waste analysis ranks food items according to their estimated waste costs and leftover quantities.

It identifies the food item with the highest estimated waste cost and the item with the highest leftover quantity. The results can guide the canteen towards reviewing its preparation practices for those items.

♦ **Demand Planning**

The demand-planning function estimates the quantity that could be prepared for each food item based on historical sales.

The program first calculates average daily sales for each food item. It then adds a 10 percent buffer and rounds the result up to the next whole portion.

Formula:

Recommended Preparation = Ceiling of (Average Daily Sales x 1.10)

The buffer is included to reduce the risk of preparing too little food. The recommendation is a basic estimate and should be reviewed in light of recent demand, holidays, weather, and special school events.

♦ **Data Visualisation Using Matplotlib**

The project generates three graphical reports:

1. A bar chart showing the estimated waste cost for each food item.
2. A comparative bar chart showing the total quantities prepared and sold for each food item.
3. A line graph showing the total quantity sold on each recorded date.

The graphs make it easier to compare items, recognise trends, and communicate the findings to canteen staff.

♦ **Exporting Reports**

The program exports three analysis reports into separate CSV files:

- daily_summary.csv
- food_item_summary.csv
- demand_plan.csv

These reports can be opened using spreadsheet software for further examination, printing, or documentation.

INTEGRATION OF PYTHON AND CSV

Python and CSV files form the foundation of the Smart School Canteen project. Python performs the calculations and analysis, while CSV provides a simple method of storing and exchanging structured data.

♦ Role of CSV

CSV stands for Comma-Separated Values. It is a plain-text file format used to store tabular data. Each line generally represents a record, while commas separate the individual fields.

The main data file contains the following fields:

- Date
- Food_Item
- Quantity_Prepared
- Quantity_Sold
- Cost_Per_Portion
- Selling_Price_Per_Portion

Each record represents the preparation and sales information for a particular food item on a particular date.

CSV is suitable for this project because it is lightweight, easy to understand, and compatible with spreadsheet applications such as Microsoft Excel and LibreOffice Calc.

♦ Reading CSV Data with Pandas

The project uses the read_csv() function from the Pandas library to load the data into a DataFrame.

Example code:

```
1. import pandas as pd
2.
3. data = pd.read_csv("canteen_data.csv")
4. print(data)
```

This code reads the CSV file and displays the records in tabular form. Once the records are loaded into a DataFrame, Python can process and analyse the information using Pandas functions.

♦ **Processing Data with Python and Pandas**

After loading the records, the program validates the data and calculates additional columns such as leftover quantity, waste percentage, estimated waste cost, sales revenue, preparation cost, and estimated net result.

Pandas allows these calculations to be applied to multiple records together. Its filtering, grouping, aggregation, and sorting functions simplify the analysis of daily sales and food-item performance.

Python also controls the menu-driven interface, accepts user input, handles errors, and coordinates the different operations of the program.

Writing Data to CSV

When the user adds a new record, the program converts the entered information into a DataFrame and appends it to the existing CSV file.

The to_csv() function is also used to export the generated analysis reports.

Example code:

```
1. import pandas as pd
2.
3. record = {
4.     "Date": ["2026-10-01"],
5.     "Food_Item": ["Sandwich"],
6.     "Quantity_Prepared": [100],
7.     "Quantity_Sold": [85],
8.     "Cost_Per_Portion": [20],
9.     "Selling_Price_Per_Portion": [30]
10. }
11.
```

```
12. data = pd.DataFrame(record)
13. data.to_csv("canteen_record.csv", index=False)
```

This example creates a DataFrame containing one record and saves it to a CSV file. The index=False parameter prevents Pandas from writing the DataFrame index as an additional column.

♦ **Role of Matplotlib**

Matplotlib works with the processed data to create graphical representations. The program uses its pyplot interface to draw bar charts and line graphs.

These charts present the calculated results in a visual format, allowing users to compare quantities, identify food items associated with higher waste costs, and observe changes in daily sales.

♦ **Overall Working Process**

The complete process follows these steps:

1. The user enters new records or uses the existing sample dataset.
2. Python reads the CSV file through Pandas.
3. The program validates the records.
4. Pandas calculates additional values and prepares summaries.
5. The program displays the analysis in tables and charts.
6. The user can generate demand recommendations.
7. The program exports the summaries and demand plan to CSV files.

The integration of these technologies demonstrates how a simple data-processing application can turn raw records into useful information for practical decision-making.

Sample Copy Prepared by: Bikramaditya Bagchi PGT (CS&IP) © 2026

SIGNIFICANCE OF THE PROJECT

The Smart School Canteen project demonstrates the importance of data analysis in improving everyday operations. It applies classroom concepts to a practical problem and shows how even a small dataset can provide useful information when it is properly organised and analysed.

- **Environmental Significance**

Reducing avoidable food waste can support more responsible use of food, water, energy, and other resources involved in food preparation. By identifying items associated with high wastage, the project encourages canteen staff to review preparation quantities and avoid unnecessary excess wherever possible.

- **Economic Significance**

The project estimates the financial value of leftover food and compares sales revenue with preparation cost. These calculations help identify possible areas for reducing losses and improving the use of available resources.

The figures are estimates based on the recorded data and do not represent a complete accounting of all canteen expenses.

- **Operational Significance**

The daily and food-item-wise summaries provide a clearer picture of canteen activities. Staff can use this information to review frequently sold items, identify items with high leftovers, and plan preparation more carefully.

The demand-planning function provides a starting point for estimating future preparation quantities. As more accurate records are collected, the canteen can compare earlier estimates with actual sales and review its planning method.

- **Educational Significance**

The project provides practical experience with the tools and concepts prescribed for Informatics Practices. Students learn how to read and write CSV files, create and manipulate DataFrames, apply calculations, filter records, group data, and create meaningful charts.

It also develops problem-solving skills, logical thinking, data interpretation, and the ability to communicate findings through reports.

- **Practical and Social Significance**

Food waste affects both household and institutional budgets. A school-based project can help students understand that responsible resource management requires observation, measurement, and informed decisions.

The same general approach can be adapted for hostels, restaurants, cafeterias, community kitchens, and other establishments where quantities prepared and quantities sold need to be monitored.

- **Future Scope**

The project can be improved further by including day-of-week analysis, weather information, special school-event records, and comparisons between predicted and actual demand. Additional reports could also identify frequently sold food items and calculate cumulative monthly trends.

A graphical user interface, automatic report generation, and improved demand-estimation methods could be introduced in future versions.

In conclusion, the Smart School Canteen project demonstrates how Python, Pandas, Matplotlib, and CSV files can be integrated to analyse food waste and support demand planning. It provides an accessible starting point for using data to make practical decisions, reduce avoidable waste, and promote responsible resource management in a school canteen.

PYTHON CODE

```
"""Smart School Canteen: Food Waste Analysis and Demand Planning

Class XII Informatics Practices project using Python, Pandas, Matplotlib and CSV.
All data is stored in canteen_data.csv in the same folder as this program.
"""

import math
import os
import re
from datetime import datetime

import pandas as pd
import matplotlib.pyplot as plt

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_FILE = os.path.join(BASE_DIR, "canteen_data.csv")
BUFFER_RATE = 0.10

REQUIRED_COLUMNS = [
    "Date",
    "Food_Item",
    "Quantity_Prepared",
    "Quantity_Sold",
    "Cost_Per_Portion",
    "Selling_Price_Per_Portion",
]

MONEY_COLUMNS = [
    "Estimated_Waste_Cost",
    "Sales_Revenue",
    "Preparation_Cost",
    "Estimated_Net_Result",
]


def ensure_data_file():
    """Create the data file with column headings if it does not exist."""
    if not os.path.exists(DATA_FILE):
        empty_data = pd.DataFrame(columns=REQUIRED_COLUMNS)
        empty_data.to_csv(DATA_FILE, index=False)
        print("Created an empty data file: " + DATA_FILE)


def load_data():
    """Read the CSV file, validate its contents, and calculate derived columns."""
    ensure_data_file()

    try:
        data = pd.read_csv(DATA_FILE)
    except Exception as error:
        raise ValueError("Could not read " + DATA_FILE + ". Details: " + str(error))

    missing_columns = [column for column in REQUIRED_COLUMNS if column not in data.columns]
    if missing_columns:
        missing_text = ", ".join(missing_columns)
        raise ValueError("The CSV file is missing these columns: " + missing_text)

    if data.empty:
        data["Date"] = pd.to_datetime(data["Date"], errors="coerce")
        data["Leftover_Quantity"] = pd.Series(dtype="int64")
        data["Waste_Percentage"] = pd.Series(dtype="float64")
        data["Estimated_Waste_Cost"] = pd.Series(dtype="float64")
        data["Sales_Revenue"] = pd.Series(dtype="float64")
        data["Preparation_Cost"] = pd.Series(dtype="float64")
        data["Estimated_Net_Result"] = pd.Series(dtype="float64")
        return data

    data["Date"] = pd.to_datetime(data["Date"], format="%Y-%m-%d", errors="coerce")
```

```
    if data["Date"].isna().any():
        raise ValueError("One or more dates are invalid. Use the format YYYY-MM-DD.")

    data["Food_Item"] = data["Food_Item"].astype(str).str.strip()
    if (data["Food_Item"] == "").any():
        raise ValueError("A food item name is blank in the CSV file.")

    number_columns = [
        "Quantity_Prepared",
        "Quantity_Sold",
        "Cost_Per_Portion",
        "Selling_Price_Per_Portion",
    ]
    for column in number_columns:
        data[column] = pd.to_numeric(data[column], errors="coerce")
        if data[column].isna().any():
            raise ValueError("The column " + column + " contains a blank or invalid number.")

    quantity_columns = ["Quantity_Prepared", "Quantity_Sold"]
    for column in quantity_columns:
        if ((data[column] % 1) != 0).any():
            raise ValueError("The column " + column + " must contain whole numbers.")

    if (data["Quantity_Prepared"] <= 0).any():
        raise ValueError("Quantity_Prepared must be greater than zero for every record.")
    if (data["Quantity_Sold"] < 0).any():
        raise ValueError("Quantity_Sold cannot be negative.")
    if (data["Quantity_Sold"] > data["Quantity_Prepared"]).any():
        raise ValueError("Quantity_Sold cannot be greater than Quantity_Prepared.")
    if (data["Cost_Per_Portion"] < 0).any():
        raise ValueError("Cost  Per  Portion cannot be negative.")
    if (data["Selling_Price_Per_Portion"] < 0).any():
        raise ValueError("Selling_Price_Per_Portion cannot be negative.")

    data["Quantity_Prepared"] = data["Quantity_Prepared"].astype(int)
    data["Quantity  Sold"] = data["Quantity  Sold"].astype(int)

    data["Leftover_Quantity"] = data["Quantity_Prepared"] - data["Quantity_Sold"]
    data["Waste_Percentage"] = (
        data["Leftover_Quantity"] / data["Quantity_Prepared"] * 100
    )
    data["Estimated_Waste_Cost"] = (
        data["Leftover_Quantity"] * data["Cost_Per_Portion"]
    )
    data["Sales_Revenue"] = data["Quantity_Sold"] * data["Selling_Price_Per_Portion"]
    data["Preparation_Cost"] = data["Quantity_Prepared"] * data["Cost_Per_Portion"]
    data["Estimated_Net_Result"] = data["Sales_Revenue"] - data["Preparation_Cost"]

    return data


def display_table(data, columns=None):
    """Print a DataFrame in a readable format with dates and money formatted."""
    if data is None or data.empty:
        print("No records to display.")
        return

    if columns is None:
        columns = list(data.columns)

    visible = data.loc[:, columns].copy()

    if "Date" in visible.columns:
        visible["Date"] = pd.to_datetime(visible["Date"], errors="coerce").dt.strftime("%Y-%m-%d")

    for column in visible.columns:
        if column in MONEY_COLUMNS or column in ["Cost_Per_Portion", "Selling_Price_Per_Portion"]:
            visible[column] = visible[column].map(
                lambda value: "" if pd.isna(value) else format(float(value), ".2f")
            )
        elif column in ["Waste_Percentage", "Average_Waste_Percentage"]:
            visible[column] = visible[column].map(
                lambda value: "" if pd.isna(value) else format(float(value), ".1f") + "%"
            )
```

```
        elif column in ["Average_Daily_Sales", "Average_Daily_Prepared", "Average_Daily_Leftover"]:
            visible[column] = visible[column].map(
                lambda value: "" if pd.isna(value) else format(float(value), ".1f")
            )

    print(visible.to_string(index=False))


def get_valid_date(prompt):
    """Ask for a date until the user enters YYYY-MM-DD correctly."""
    while True:
        date_text = input(prompt).strip()
        try:
            parsed_date = datetime.strptime(date_text, "%Y-%m-%d")
            return parsed_date.strftime("%Y-%m-%d")
        except ValueError:
            print("Invalid date. Enter the date in YYYY-MM-DD format, for example 2026-09-01.")


def get_integer(prompt, minimum=0):
    """Read a whole number and enforce a minimum value."""
    while True:
        raw_value = input(prompt).strip()
        try:
            value = int(raw_value)
        except ValueError:
            print("Enter a whole number.")
            continue

        if value < minimum:
            print("Enter a value greater than or equal to " + str(minimum) + ".")
            continue
        return value


def get  number(prompt, minimum=0.0):
    """Read a number and enforce a minimum value."""
    while True:
        raw_value = input(prompt).strip()
        try:
            value = float(raw  value)
        except ValueError:
            print("Enter a valid number. Decimals are allowed.")
            continue

        if value < minimum:
            print("Enter a value greater than or equal to " + str(minimum) + ".")
            continue
        return value


def view_all_records():
    """Display the complete canteen dataset and calculated results."""
    try:
        data = load_data()
    except ValueError as error:
        print("Data error: " + str(error))
        return

    if data.empty:
        print("The data file is empty. Add a record first.")
        return

    data = data.sort_values(["Date", "Food_Item"])
    columns = [
        "Date",
        "Food_Item",
        "Quantity_Prepared",
        "Quantity_Sold",
        "Leftover_Quantity",
        "Waste_Percentage",
        "Estimated_Waste_Cost",
        "Sales_Revenue",
        "Estimated_Net_Result",
```

```
    ]
    display_table(data, columns)
    print("\nNote: All money values are in the same currency unit entered in the CSV file.")


def add_record():
    """Collect one food-item record and append it to the CSV file."""
    try:
        current_data = load_data()
    except ValueError as error:
        print("Data error: " + str(error))
        return

    print("\nAdd a Canteen Record")
    date_text = get_valid_date("Date (YYYY-MM-DD): ")

    while True:
        food_item = input("Food item name: ").strip()
        if food_item:
            food_item = food_item.title()
            break
        print("Food item name cannot be blank.")

    prepared = get_integer("Quantity prepared (whole portions, at least 1): ", 1)
    sold = get_integer("Quantity sold (0 to quantity prepared): ", 0)
    while sold > prepared:
        print("Quantity sold cannot be greater than quantity prepared.")
        sold = get_integer("Quantity sold (0 to " + str(prepared) + "): ", 0)

    cost = get_number("Cost to prepare one portion: ", 0.0)
    price = get  number("Selling price of one portion: ", 0.0)

    new_record = {
        "Date": date_text,
        "Food_Item": food_item,
        "Quantity  Prepared": prepared,
        "Quantity_Sold": sold,
        "Cost_Per_Portion": cost,
        "Selling_Price_Per_Portion": price,
    }

    file_exists = os.path.exists(DATA_FILE) and os.path.getsize(DATA_FILE) > 0
    try:
        pd.DataFrame([new_record], columns=REQUIRED_COLUMNS).to_csv(
            DATA_FILE,
            mode="a",
            header=not file_exists,
            index=False,
        )
        print("Record saved successfully in " + DATA_FILE + ".")
        print("Leftover quantity for this record: " + str(prepared - sold))
    except Exception as error:
        print("Could not save the record. Details: " + str(error))


def search_records():
    """Search records by an optional date and/or part of a food item name."""
    try:
        data = load_data()
    except ValueError as error:
        print("Data error: " + str(error))
        return

    if data.empty:
        print("The data file is empty. Add a record first.")
        return

    print("Leave a search field blank to skip that filter.")
    date_text = input("Date (YYYY-MM-DD), or press Enter to skip: ").strip()
    item_text = input("Food item name or part of name, or press Enter to skip: ").strip()

    filtered = data.copy()
    if date_text:
        try:
```

```
            parsed_date = datetime.strptime(date_text, "%Y-%m-%d").strftime("%Y-%m-%d")
            filtered = filtered[filtered["Date"].dt.strftime("%Y-%m-%d") == parsed_date]
        except ValueError:
            print("Invalid date. Use YYYY-MM-DD.")
            return

    if item_text:
        filtered = filtered[
            filtered["Food_Item"].str.contains(re.escape(item_text), case=False, na=False)
        ]

    if filtered.empty:
        print("No matching records were found.")
        return

    filtered = filtered.sort_values(["Date", "Food_Item"])
    columns = [
        "Date",
        "Food_Item",
        "Quantity_Prepared",
        "Quantity_Sold",
        "Leftover_Quantity",
        "Waste_Percentage",
        "Estimated_Waste_Cost",
    ]
    display_table(filtered, columns)


def make_summary(data, group_column):
    """Create totals grouped by date or food item."""
    label_column = "Food_Item" if group_column == "Date" else "Date"
    summary = data.groupby(group_column, as_index=False).agg(
        Record_Count=(label_column, "count"),
        Total_Prepared=("Quantity_Prepared", "sum"),
        Total_Sold=("Quantity_Sold", "sum"),
        Total_Leftover=("Leftover_Quantity", "sum"),
        Estimated_Waste_Cost=("Estimated_Waste_Cost", "sum"),
        Sales_Revenue=("Sales_Revenue", "sum"),
        Preparation_Cost=("Preparation_Cost", "sum"),
        Estimated_Net_Result=("Estimated_Net_Result", "sum"),
    )
    summary["Waste_Percentage"] = (
        summary["Total_Leftover"] / summary["Total_Prepared"] * 100
    )
    return summary


def daily_summary():
    """Display a daily total for prepared, sold, leftover and estimated money values."""
    try:
        data = load_data()
    except ValueError as error:
        print("Data error: " + str(error))
        return

    if data.empty:
        print("The data file is empty. Add a record first.")
        return

    summary = make_summary(data, "Date").sort_values("Date")
    display_table(
        summary,
        [
            "Date",
            "Record_Count",
            "Total_Prepared",
            "Total_Sold",
            "Total_Leftover",
            "Waste_Percentage",
            "Estimated_Waste_Cost",
            "Sales_Revenue",
            "Estimated_Net_Result",
        ],
    )
```

```
365.
366.
367. def food_item_summary():
368.     """Display totals grouped by food item."""
369.     try:
370.         data = load_data()
371.     except ValueError as error:
372.         print("Data error: " + str(error))
373.         return
374.
375.     if data.empty:
376.         print("The data file is empty. Add a record first.")
377.         return
378.
379.     summary = make_summary(data, "Food_Item").sort_values("Total_Sold", ascending=False)
380.     display_table(
381.         summary,
382.         [
383.             "Food_Item",
384.             "Record_Count",
385.             "Total_Prepared",
386.             "Total_Sold",
387.             "Total_Leftover",
388.             "Waste_Percentage",
389.             "Estimated_Waste_Cost",
390.             "Sales_Revenue",
391.             "Estimated_Net_Result",
392.         ],
393.     )
394.
395.
396. def waste_analysis():
397.     """Rank food items by estimated waste cost and quantity left over."""
398.     try:
399.         data = load_data()
400.     except ValueError as error:
401.         print("Data error: " + str(error))
402.         return
403.
404.     if data.empty:
405.         print("The data file is empty. Add a record first.")
406.         return
407.
408.     summary = make_summary(data, "Food_Item")
409.     summary = summary.sort_values(
410.         ["Estimated_Waste_Cost", "Total_Leftover"], ascending=[False, False]
411.     )
412.
413.     print("\nFood Waste Analysis")
414.     display_table(
415.         summary,
416.         [
417.             "Food_Item",
418.             "Total_Prepared",
419.             "Total_Sold",
420.             "Total_Leftover",
421.             "Waste_Percentage",
422.             "Estimated_Waste_Cost",
423.         ],
424.     )
425.
426.     highest_cost_item = summary.iloc[0]
427.     highest_quantity_item = summary.sort_values("Total_Leftover", ascending=False).iloc[0]
428.     print("\nItem with the highest estimated waste cost: " + str(highest_cost_item["Food_Item"]))
429.     print("Estimated waste cost: " + format(float(highest_cost_item["Estimated_Waste_Cost"]), ".2f"))
430.     print("Item with the highest leftover quantity: " + str(highest_quantity_item["Food_Item"]))
431.     print("Leftover portions: " + str(int(highest_quantity_item["Total_Leftover"])))
432.     print("Recommendation: review preparation quantities, portion sizes, and demand for these items.")
433.
434.
435. def build_demand_plan(data):
436.     """Estimate future preparation quantities using recorded average daily sales."""
437.     daily_item = data.groupby(["Date", "Food_Item"], as_index=False).agg(
438.         Quantity_Prepared=("Quantity_Prepared", "sum"),
```

```
        Quantity_Sold=("Quantity_Sold", "sum"),
        Leftover_Quantity=("Leftover_Quantity", "sum"),
    )

    plan = daily_item.groupby("Food_Item", as_index=False).agg(
        Days_Recorded=("Date", "nunique"),
        Average_Daily_Sales=("Quantity_Sold", "mean"),
        Average_Daily_Prepared=("Quantity_Prepared", "mean"),
        Average_Daily_Leftover=("Leftover_Quantity", "mean"),
        Total_Prepared=("Quantity_Prepared", "sum"),
        Total_Sold=("Quantity_Sold", "sum"),
        Total_Leftover=("Leftover_Quantity", "sum"),
    )

    plan["Average_Waste_Percentage"] = (
        plan["Total_Leftover"] / plan["Total_Prepared"] * 100
    )
    plan["Recommended_Preparation"] = plan["Average_Daily_Sales"].apply(
        lambda average: int(math.ceil(float(average) * (1 + BUFFER_RATE)))
    )
    plan["Buffer_Percentage"] = BUFFER_RATE * 100
    return plan.sort_values("Average_Daily_Sales", ascending=False)


def demand_planning():
    """Show estimated preparation quantities using average recorded sales plus a buffer."""
    try:
        data = load_data()
    except ValueError as error:
        print("Data error: " + str(error))
        return

    if data.empty:
        print("The data file is empty. Add a record first.")
        return

    plan = build_demand_plan(data)
    print("\nDemand Planning Report")
    display_table(
        plan,
        [
            "Food_Item",
            "Days_Recorded",
            "Average_Daily_Sales",
            "Average_Daily_Prepared",
            "Average_Daily_Leftover",
            "Average_Waste_Percentage",
            "Recommended_Preparation",
        ],
    )
    print("\nThe recommended quantity is average recorded sales plus a 10 percent buffer, rounded up.")
    print("This is a simple estimate. Review holidays, weather, school events and recent demand before using it.")
    print("Record every item consistently to make the average more useful.")


def export_reports():
    """Export daily summaries, item summaries and the demand plan to separate CSV files."""
    try:
        data = load_data()
    except ValueError as error:
        print("Data error: " + str(error))
        return

    if data.empty:
        print("The data file is empty. Add a record before exporting reports.")
        return

    daily = make_summary(data, "Date").sort_values("Date")
    items = make_summary(data, "Food_Item").sort_values("Total_Sold", ascending=False)
    plan = build_demand_plan(data)

    try:
        daily.to_csv(os.path.join(BASE_DIR, "daily_summary.csv"), index=False, date_format="%Y-%m-%d")
        items.to_csv(os.path.join(BASE_DIR, "food_item_summary.csv"), index=False)
```

```
        plan.to_csv(os.path.join(BASE_DIR, "demand_plan.csv"), index=False)
        print("Reports exported successfully:")
        print("1. daily_summary.csv")
        print("2. food_item_summary.csv")
        print("3. demand_plan.csv")
    except Exception as error:
        print("Could not export reports. Details: " + str(error))


def chart_food_waste():
    """Plot estimated waste cost by food item."""
    try:
        data = load_data()
    except ValueError as error:
        print("Data error: " + str(error))
        return

    if data.empty:
        print("The data file is empty. Add a record first.")
        return

    summary = make_summary(data, "Food_Item").sort_values("Estimated_Waste_Cost", ascending=False)
    plt.figure(figsize=(10, 6))
    plt.bar(summary["Food_Item"], summary["Estimated_Waste_Cost"])
    plt.title("Estimated Food Waste Cost by Item")
    plt.xlabel("Food Item")
    plt.ylabel("Estimated Waste Cost")
    plt.xticks(rotation=35, ha="right")
    plt.tight_layout()
    plt.show()
    plt.close()


def chart_prepared_vs_sold():
    """Plot total quantities prepared and sold for each food item."""
    try:
        data = load_data()
    except ValueError as error:
        print("Data error: " + str(error))
        return

    if data.empty:
        print("The data file is empty. Add a record first.")
        return

    summary = make_summary(data, "Food_Item").sort_values("Total_Prepared", ascending=False)
    positions = list(range(len(summary)))
    bar_width = 0.38

    plt.figure(figsize=(10, 6))
    plt.bar(
        [position - bar_width / 2 for position in positions],
        summary["Total_Prepared"],
        width=bar_width,
        label="Prepared",
    )
    plt.bar(
        [position + bar_width / 2 for position in positions],
        summary["Total_Sold"],
        width=bar_width,
        label="Sold",
    )
    plt.title("Total Quantity Prepared and Sold by Food Item")
    plt.xlabel("Food Item")
    plt.ylabel("Quantity in Portions")
    plt.xticks(positions, summary["Food_Item"], rotation=35, ha="right")
    plt.legend()
    plt.tight_layout()
    plt.show()
    plt.close()


def chart_daily_sales_trend():
    """Plot total portions sold each day."""
```

```
    try:
        data = load_data()
    except ValueError as error:
        print("Data error: " + str(error))
        return

    if data.empty:
        print("The data file is empty. Add a record first.")
        return

    summary = make_summary(data, "Date").sort_values("Date")
    date_labels = pd.to_datetime(summary["Date"]).dt.strftime("%Y-%m-%d")

    plt.figure(figsize=(10, 6))
    plt.plot(date_labels, summary["Total_Sold"], marker="o", label="Portions Sold")
    plt.title("Daily Canteen Sales Trend")
    plt.xlabel("Date")
    plt.ylabel("Total Portions Sold")
    plt.xticks(rotation=45, ha="right")
    plt.legend()
    plt.tight_layout()
    plt.show()
    plt.close()


def show_menu():
    """Display the program menu."""
    print("\nSMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING")
    print("=" * 64)
    print("1. View all records")
    print("2. Add a new record")
    print("3. Search records")
    print("4. Show daily summary")
    print("5. Show food item summary")
    print("6. Analyse food waste")
    print("7. Create demand planning report")
    print("8. Export summary reports to CSV")
    print("9. Chart: Food waste cost by item")
    print("10. Chart: Prepared versus sold")
    print("11. Chart: Daily sales trend")
    print("0. Exit")
    print("=" * 64)


def main():
    """Run the menu-driven application."""
    ensure_data_file()

    actions = {
        "1": view_all_records,
        "2": add_record,
        "3": search_records,
        "4": daily_summary,
        "5": food_item_summary,
        "6": waste_analysis,
        "7": demand_planning,
        "8": export_reports,
        "9": chart_food_waste,
        "10": chart_prepared_vs_sold,
        "11": chart_daily_sales_trend,
    }

    while True:
        show_menu()
        choice = input("Enter your choice: ").strip()

        if choice == "0":
            print("Thank you for using the Smart School Canteen project.")
            break
        elif choice in actions:
            actions[choice]()
        else:
            print("Invalid choice. Enter a number from 0 to 11.")

```

```
661.
662. if __name__ == "__main__":
663.     try:
664.         main()
665.     except KeyboardInterrupt:
666.         print("\nProgram stopped by the user.")
```

OUTPUT:

```
1. SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING
2. ============================================================
3. 1. View all records
4. 2. Add a new record
5. 3. Search records
6. 4. Show daily summary
7. 5. Show food item summary
8. 6. Analyse food waste
9. 7. Create demand planning report
10. 8. Export summary reports to CSV
11. 9. Chart: Food waste cost by item
12. 10. Chart: Prepared versus sold
13. 11. Chart: Daily sales trend
14. 0. Exit
15. ============================================================
16. Enter your choice: 1
17.       Date  Food_Item  Quantity_Prepared  Quantity_Sold  Leftover_Quantity Waste_Percentage Estimated_Waste_Cost
Sales_Revenue Estimated_Net_Result
18. 2026-09-01   Fruit Cup                 20             14                  6            30.0%               72.00          280.00               40.00
19. 2026-09-01   Idli Plate                35             28                  7            20.0%               70.00          504.00              154.00
20. 2026-09-01   Rice Plate                45             42                  3             6.7%               75.00         1680.00              555.00
21. 2026-09-01      Samosa                 50             46                  4             8.0%               24.00          460.00              160.00
22. 2026-09-01 Veg Sandwich                30             22                  8            26.7%              144.00          660.00              120.00
23. 2026-09-02   Fruit Cup                 22             16                  6            27.3%               72.00          320.00               56.00
24. 2026-09-02   Idli Plate                34             27                  7            20.6%               70.00          486.00              146.00
25. 2026-09-02   Rice Plate                44             40                  4             9.1%              100.00         1600.00              500.00
26. 2026-09-02      Samosa                 48             45                  3             6.2%               18.00          450.00              162.00
27. 2026-09-02 Veg Sandwich                28             23                  5            17.9%               90.00          690.00              186.00
28. 2026-09-03   Fruit Cup                 20             15                  5            25.0%               60.00          300.00               60.00
29. 2026-09-03   Idli Plate                36             31                  5            13.9%               50.00          558.00              198.00
30. 2026-09-03   Rice Plate                46             43                  3             6.5%               75.00         1720.00              570.00
31. 2026-09-03      Samosa                 52             49                  3             5.8%               18.00          490.00              178.00
32. 2026-09-03 Veg Sandwich                32             26                  6            18.8%              108.00          780.00              204.00
33. 2026-09-04   Fruit Cup                 18             12                  6            33.3%               72.00          240.00               24.00
34. 2026-09-04   Idli Plate                32             24                  8            25.0%               80.00          432.00              112.00
35. 2026-09-04   Rice Plate                42             39                  3             7.1%               75.00         1560.00              510.00
36. 2026-09-04      Samosa                 50             43                  7            14.0%               42.00          430.00              130.00
37. 2026-09-04 Veg Sandwich                30             21                  9            30.0%              162.00          630.00               90.00
38. 2026-09-07   Fruit Cup                 16             11                  5            31.2%               60.00          220.00               28.00
39. 2026-09-07   Idli Plate                30             23                  7            23.3%               70.00          414.00              114.00
40. 2026-09-07   Rice Plate                40             38                  2             5.0%               50.00         1520.00              520.00
41. 2026-09-07      Samosa                 45             42                  3             6.7%               18.00          420.00              150.00
42. 2026-09-07 Veg Sandwich                26             20                  6            23.1%              108.00          600.00              132.00
43. 2026-09-08   Fruit Cup                 20             15                  5            25.0%               60.00          300.00               60.00
44. 2026-09-08   Idli Plate                34             29                  5            14.7%               50.00          522.00              182.00
45. 2026-09-08   Rice Plate                43             41                  2             4.7%               50.00         1640.00              565.00
46. 2026-09-08      Samosa                 48             46                  2             4.2%               12.00          460.00              172.00
47. 2026-09-08 Veg Sandwich                29             24                  5            17.2%               90.00          720.00              198.00
48. 2026-09-09   Fruit Cup                 21             16                  5            23.8%               60.00          320.00               68.00
49. 2026-09-09   Idli Plate                35             30                  5            14.3%               50.00          540.00              190.00
50. 2026-09-09   Rice Plate                45             42                  3             6.7%               75.00         1680.00              555.00
51. 2026-09-09      Samosa                 50             47                  3             6.0%               18.00          470.00              170.00
52. 2026-09-09 Veg Sandwich                31             25                  6            19.4%              108.00          750.00              192.00
53. 2026-09-10   Fruit Cup                 18             10                  8            44.4%               96.00          200.00              -16.00
54. 2026-09-10   Idli Plate                31             22                  9            29.0%               90.00          396.00               86.00
55. 2026-09-10   Rice Plate                41             38                  3             7.3%               75.00         1520.00              495.00
56. 2026-09-10      Samosa                 46             40                  6            13.0%               36.00          400.00              124.00
57. 2026-09-10 Veg Sandwich                28             19                  9            32.1%              162.00          570.00               66.00
58.
59. Note: All money values are in the same currency unit entered in the CSV file.
60.
61. SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING
62. ============================================================
63. 1. View all records
```

2. Add a new record
3. Search records
4. Show daily summary
5. Show food item summary
6. Analyse food waste
7. Create demand planning report
8. Export summary reports to CSV
9. Chart: Food waste cost by item
10. Chart: Prepared versus sold
11. Chart: Daily sales trend
0. Exit

==========

Enter your choice: 3
Leave a search field blank to skip that filter.
Date (YYYY-MM-DD), or press Enter to skip: 2026-04-01
Food item name or part of name, or press Enter to skip:
No matching records were found.

SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING

==========

1. View all records
2. Add a new record
3. Search records
4. Show daily summary
5. Show food item summary
6. Analyse food waste
7. Create demand planning report
8. Export summary reports to CSV
9. Chart: Food waste cost by item
10. Chart: Prepared versus sold
11. Chart: Daily sales trend
0. Exit

==========

Enter your choice: 3
Leave a search field blank to skip that filter.
Date (YYYY-MM-DD), or press Enter to skip: 2026-09-01
Food item name or part of name, or press Enter to skip:

Date	Food_Item	Quantity_Prepared	Quantity_Sold	Leftover_Quantity	Waste_Percentage	Estimated_Waste_Cost
2026-09-01	Fruit Cup	20	14	6	30.0%	72.00
2026-09-01	Idli Plate	35	28	7	20.0%	70.00
2026-09-01	Rice Plate	45	42	3	6.7%	75.00
2026-09-01	Samosa	50	46	4	8.0%	24.00
2026-09-01	Veg Sandwich	30	22	8	26.7%	144.00

SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING

==========

1. View all records
2. Add a new record
3. Search records
4. Show daily summary
5. Show food item summary
6. Analyse food waste
7. Create demand planning report
8. Export summary reports to CSV
9. Chart: Food waste cost by item
10. Chart: Prepared versus sold
11. Chart: Daily sales trend
0. Exit

==========

Enter your choice: 4

Date	Record_Count	Total_Prepared	Total_Sold	Total_Leftover	Waste_Percentage	Estimated_Waste_Cost	Sales_Revenue	Estimated_Net_Result
2026-09-01	5	180	152	28	15.6%	385.00	3584.00	1029.00
2026-09-02	5	176	151	25	14.2%	350.00	3546.00	1050.00
2026-09-03	5	186	164	22	11.8%	311.00	3848.00	1210.00
2026-09-04	5	172	139	33	19.2%	431.00	3292.00	866.00
2026-09-07	5	157	134	23	14.6%	306.00	3174.00	944.00
2026-09-08	5	174	155	19	10.9%	262.00	3642.00	1177.00
2026-09-09	5	182	160	22	12.1%	311.00	3760.00	1175.00
2026-09-10	5	164	129	35	21.3%	459.00	3086.00	755.00

SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING

==========

1. View all records

2. **Add** a new record
3. **Search** records
4. **Show** daily summary
5. **Show** food item summary
6. **Analyse** food waste
7. **Create** demand planning report
8. **Export** summary reports to CSV
9. **Chart**: **Food** waste cost by item
10. **Chart**: **Prepared** versus sold
11. **Chart**: **Daily** sales trend
0. **Exit**

==========

Enter your choice: 5

Food_Item	Record_Count	Total_Prepared	Total_Sold	Total_Leftover	Waste_Percentage	Estimated_Waste_Cost	Sales_Revenue	Estimated_Net_Result
Samosa	8	389	358	31	8.0%	186.00	3580.00	1246.00
Rice Plate	8	346	323	23	6.6%	575.00	12920.00	4270.00
Idli Plate	8	267	214	53	19.9%	530.00	3852.00	1182.00
Veg Sandwich	8	234	180	54	23.1%	972.00	5400.00	1188.00
Fruit Cup	8	155	109	46	29.7%	552.00	2180.00	320.00

SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING

==========

1. **View** all records
2. **Add** a new record
3. **Search** records
4. **Show** daily summary
5. **Show** food item summary
6. **Analyse** food waste
7. **Create** demand planning report
8. **Export** summary reports to CSV
9. **Chart**: **Food** waste cost by item
10. **Chart**: **Prepared** versus sold
11. **Chart**: **Daily** sales trend
0. **Exit**

==========

Enter your choice: 6

Food Waste Analysis

Food_Item	Total_Prepared	Total_Sold	Total_Leftover	Waste_Percentage	Estimated_Waste_Cost
Veg Sandwich	234	180	54	23.1%	972.00
Rice Plate	346	323	23	6.6%	575.00
Fruit Cup	155	109	46	29.7%	552.00
Idli Plate	267	214	53	19.9%	530.00
Samosa	389	358	31	8.0%	186.00

Item with the highest estimated waste cost: **Veg Sandwich**

Estimated waste cost: 972.00

Item with the highest leftover quantity: **Veg Sandwich**

Leftover portions: 54

Recommendation: review preparation quantities, portion sizes, and demand for these items.

SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING

==========

1. **View** all records
2. **Add** a new record
3. **Search** records
4. **Show** daily summary
5. **Show** food item summary
6. **Analyse** food waste
7. **Create** demand planning report
8. **Export** summary reports to CSV
9. **Chart**: **Food** waste cost by item
10. **Chart**: **Prepared** versus sold
11. **Chart**: **Daily** sales trend
0. **Exit**

==========

Enter your choice: 7

Demand Planning Report

Food_Item	Days_Recorded	Average_Daily_Sales	Average_Daily_Prepared	Average_Daily_Leftover	Average_Waste_Percentage	Recommended_Preparation
Samosa	8	44.8	48.6	3.9	8.0%	50
Rice Plate	8	40.4	43.2	2.9	6.6%	45

```
  Idli Plate          8        26.8        33.4        6.6        19.9%          30
Veg Sandwich          8        22.5        29.2        6.8        23.1%          25
  Fruit Cup           8        13.6        19.4        5.8        29.7%          15

The recommended quantity is average recorded sales plus a 10 percent buffer, rounded up.
This is a simple estimate. Review holidays, weather, school events and recent demand before using it.
Record every item consistently to make the average more useful.

SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING
==============================================================
1. View all records
2. Add a new record
3. Search records
4. Show daily summary
5. Show food item summary
6. Analyse food waste
7. Create demand planning report
8. Export summary reports to CSV
9. Chart: Food waste cost by item
10. Chart: Prepared versus sold
11. Chart: Daily sales trend
0. Exit
==============================================================
Enter your choice: 8
Reports exported successfully:
1. daily_summary.csv
2. food_item_summary.csv
3. demand_plan.csv

SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING
==============================================================
1. View all records
2. Add a new record
3. Search records
4. Show daily summary
5. Show food item summary
6. Analyse food waste
7. Create demand planning report
8. Export summary reports to CSV
9. Chart: Food waste cost by item
10. Chart: Prepared versus sold
11. Chart: Daily sales trend
0. Exit
==============================================================
Enter your choice: 9

SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING
==============================================================
1. View all records
2. Add a new record
3. Search records
4. Show daily summary
5. Show food item summary
6. Analyse food waste
7. Create demand planning report
8. Export summary reports to CSV
9. Chart: Food waste cost by item
10. Chart: Prepared versus sold
11. Chart: Daily sales trend
0. Exit
==============================================================
Enter your choice: 10

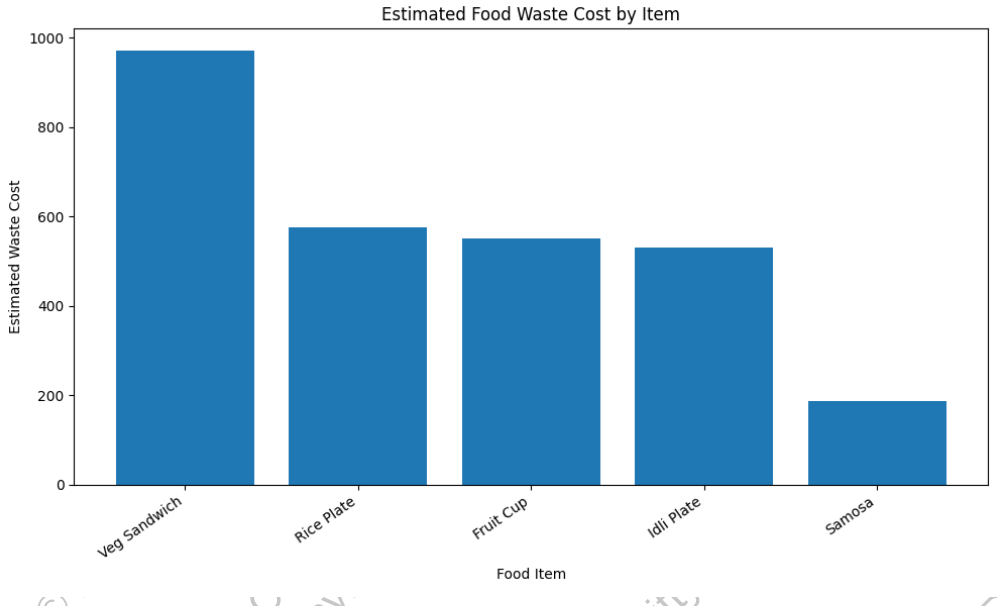
SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING
==============================================================
1. View all records
2. Add a new record
3. Search records
4. Show daily summary
5. Show food item summary
6. Analyse food waste
7. Create demand planning report
8. Export summary reports to CSV
9. Chart: Food waste cost by item
```

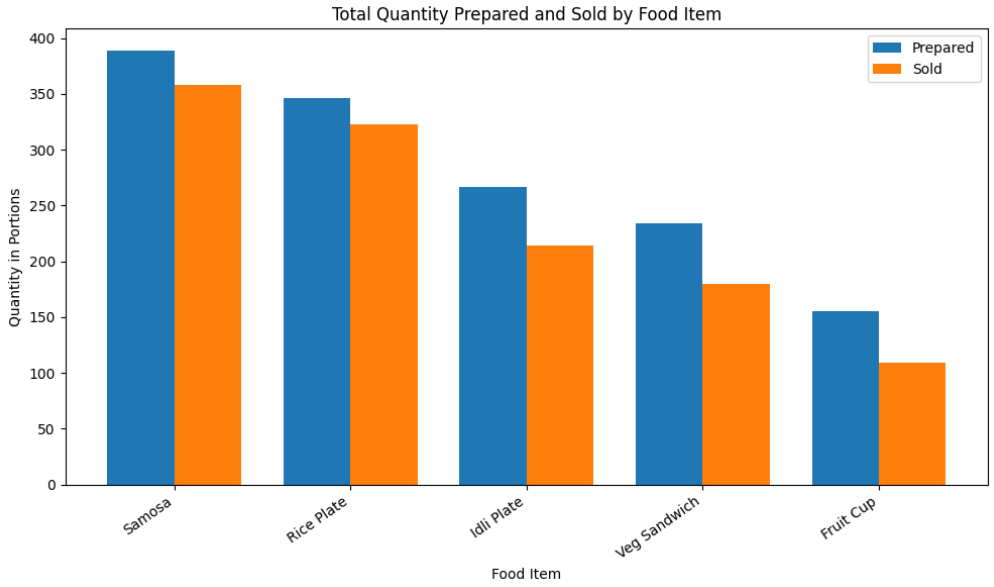
```
10. Chart: Prepared versus sold
11. Chart: Daily sales trend
0. Exit
==========
Enter your choice: 11

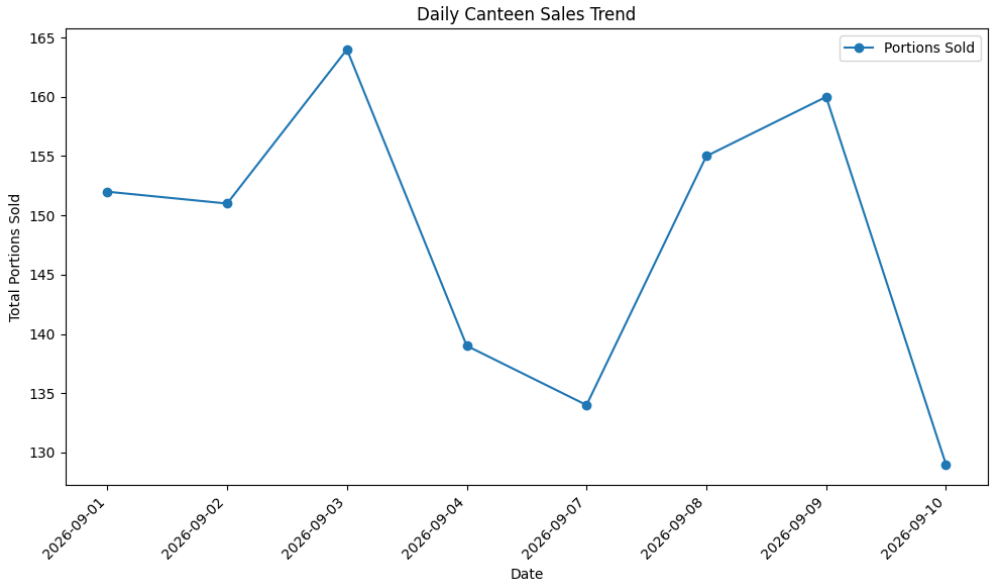
SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING
==========
1. View all records
2. Add a new record
3. Search records
4. Show daily summary
5. Show food item summary
6. Analyse food waste
7. Create demand planning report
8. Export summary reports to CSV
9. Chart: Food waste cost by item
10. Chart: Prepared versus sold
11. Chart: Daily sales trend
0. Exit
==========
Enter your choice: 9

SMART SCHOOL CANTEEN: FOOD WASTE ANALYSIS AND DEMAND PLANNING
==========
1. View all records
2. Add a new record
3. Search records
4. Show daily summary
5. Show food item summary
6. Analyse food waste
7. Create demand planning report
8. Export summary reports to CSV
9. Chart: Food waste cost by item
10. Chart: Prepared versus sold
11. Chart: Daily sales trend
0. Exit
==========
Enter your choice: 0
```

Graphs:


Estimated Food Waste Cost by Item
Estimated Waste Cost
Food Item
Veg Sandwich
Rice Plate
Fruit Cup
Idli Plate
Samosa


Total Quantity Prepared and Sold by Food Item
Prepared
Sold
Quantity in Portions
Food Item
Samosa
Rice Plate
Idli Plate
Veg Sandwich
Fruit Cup


Daily Canteen Sales Trend
Portions Sold
Total Portions Sold
Date
2026-09-01
2026-09-02
2026-09-03
2026-09-04
2026-09-07
2026-09-08
2026-09-09
2026-09-10

BIBLIOGRAPHY

Sl.	Description	Source
1.	Python	[https://python.org]
2.	Python – Pandas	[https://pandas.python.org]
3.	Informatics Practices – A textbook for Class XI	Sumita Arora
4.	Informatics Practices – A textbook for Class XII	Sumita Arora
5.	GeeksforGeeks	[https://geeksforgeeks.com]
6.	PYNative	[https://pynative.com]

END OF DOCUMENT

Suggested Projects:

1. Energy Detective: Analysing Electricity Consumption and Saving Costs

Real-world problem: Families and schools often know that electricity bills are high but do not track how consumption changes over time.

Data to collect in CSV: Month, electricity meter reading, units consumed, tariff per unit, and electricity bill. Appliance-wise readings can be added if available.

What students will analyse: Which months have the highest consumption? How much does consumption change between months? What would be the estimated saving from a chosen reduction in usage?

Pandas and Matplotlib: Calculate monthly consumption and estimated costs, compare periods, and visualise consumption using line graphs and monthly bills using bar charts.

Practical outcome: A simple energy-saving report with estimated costs and realistic targets for reducing usage.

Data source: Household or school electricity bills, with permission and personal details removed.

2. Smart Grocery Budget Planner: Tracking Essential Commodity Prices

Real-world problem: Families may struggle to manage grocery budgets when prices vary across shops or change from week to week.

Data to collect in CSV: Date, commodity, quantity, unit, shop or market, price, and planned purchase quantity.

What students will analyse: Which items have increased in price? Which shop offers the lowest total cost for a fixed basket? How much would the same basket cost in different months?

Pandas and Matplotlib: Calculate basket totals, compare shops, find price changes, and plot commodity prices over time with line graphs and comparative bar charts.

Practical outcome: A price-comparison report that helps a household plan purchases and stay within a budget.

Data source: A student-conducted local market survey or publicly available price records. Compare equivalent units and product grades.

3. Library Insights: Understanding Book Demand and Improving Reading Resources

Real-world problem: School libraries may buy books that are rarely borrowed while popular books remain unavailable.

Data to collect in CSV: Book ID, title, genre or subject, issue date, return date, and availability status. Borrower names are unnecessary.

What students will analyse: Which genres are most popular? Which books are frequently borrowed? Which resources remain unused? During which months is library use highest?

Pandas and Matplotlib: Count issues by category, calculate borrowing frequency, group activity by month, and generate bar charts, line graphs, and histograms of borrowing duration.

Practical outcome: Suggestions for new book purchases, better placement of resources, and reading-promotion activities.

Data source: An anonymised library issue register or a sample dataset compiled with the librarian's permission.

4. Every Drop Counts: Rainwater Harvesting Potential Calculator

Real-world problem: Schools and homes may receive substantial rainfall but fail to estimate how much water could be collected and stored.

Data to collect in CSV: Month, rainfall in millimetres, roof area in square metres, assumed runoff coefficient, and available storage capacity.

What students will analyse: Which months offer the greatest harvesting potential? How much water could be collected annually? Would the proposed tank capacity be sufficient for the estimated collection?

Pandas and Matplotlib: Calculate monthly harvest estimates, aggregate annual totals, compare rainfall and collection potential, and plot monthly rainfall and harvest estimates.

Practical outcome: A preliminary, data-based plan for rainwater harvesting at school or home.

A useful calculation: Estimated harvest in litres = rainfall (mm) × roof area (m^2) × runoff coefficient. The coefficient accounts approximately for water lost before collection.

Data source: Local rainfall records and measured or documented roof dimensions. Clearly state any assumptions.

5. On-Time Journey: Analysing Public Transport Delays and Travel Costs

Real-world problem: Students and commuters may face unpredictable travel times and difficulty deciding which public transport option is more reliable.

Data to collect in CSV: Route label, travel date, scheduled arrival time, actual arrival time, journey duration, fare, and mode of transport.

What students will analyse: Which routes experience the largest delays? Are delays more frequent on certain days or at particular times? Which observed option offers the best balance of cost and punctuality?

Pandas and Matplotlib: Calculate delay duration, average delays, route-wise costs, and punctuality percentages. Display findings with bar charts, line graphs, and histograms.

Practical outcome: An evidence-based travel comparison to help commuters plan more reliably.

Data source: A short travel observation study or public timetable data. Use anonymous route labels and avoid collecting students' home addresses.

6. Computer Lab Monitor: Equipment Faults, Downtime and Maintenance Planning

Real-world problem: Computer laboratories lose useful working time when recurring faults are not recorded or maintenance is delayed.

Data to collect in CSV: Device ID, device type, fault category, date reported, date repaired, downtime, repair cost, and current status.

What students will analyse: Which devices fail most often? What are the most common problems? Which faults cause the longest downtime? How does repair spending change over time?

Pandas and Matplotlib: Count faults by category, calculate repair duration, aggregate downtime and cost, and plot fault frequency, monthly repair spending, and device-wise downtime.

Practical outcome: A maintenance summary that helps laboratory staff prioritise repairs and identify recurring problems.

Data source: An approved, anonymised maintenance register. A simulated dataset is acceptable for demonstrating the code if real records are unavailable, provided it is clearly labelled as simulated.

Sample Copy Prepared by: Bikramaditya Bagchi PGT (CS&IP) © 2026